

AI proposes. *Rules decide.* People approve.

A complete, plain-language explanation of what ProvenPath is, how every piece of the pipeline works, and what every word on the screen means, from the first prompt to a product confirmed inside Guidewire PolicyCenter.

1. The idea in one page
2. The pipeline, end to end
3. The worked example: the ₹50L cyber product
4. The deterministic gate: 6 layers, 23 rules
5. Trust and tamper-proofing: hashes, tokens, signatures
6. From approval to PolicyCenter: package, agent, restart, confirmation
7. The system: components, modules, where everything runs
8. Every screen and every word in the frontend
9. Insurance and Guidewire glossary
10. Technical glossary
11. Reference: events and API endpoints
12. What is real today, what is not, and how to run it

Written 29 Sep 2026 from the code on `main` plus the Track B work (package builder, PC agent, VM scripts).

The idea in one page

The problem

An insurance **product** (for example "cyber insurance for small Indian companies") is a precise configuration: which coverages exist, how much each one pays at most (**limits**), what the customer pays first (**deductibles**), what is excluded, which conditions apply, how it is priced. In India that configuration must follow the insurance regulator's (IRDAI) rules and CERT-In's cyber-incident rules. Building one by hand in Guidewire PolicyCenter takes weeks (our PRD's baseline estimate is about three weeks).

A large language model (LLM) can draft the same configuration in seconds. But an LLM is fluent, confident and sometimes wrong: it may set a sublimit above the legal cap, forget a mandatory exclusion, or "paraphrase" a regulation into something the regulator never wrote. In a regulated product, "sometimes wrong" is not acceptable.

The answer: a pre-commit compliance gate

ProvenPath sits **between the AI and PolicyCenter**. Nothing the AI writes can reach PolicyCenter unless three things happen, in order:

1. **AI proposes.** A planner (Google Gemini) turns a plain-language request into a structured **proposal**: PolicyCenter coverage patterns with limits, deductibles, waiting hours, exclusions and conditions, each clause citing the regulation it relies on.
2. **Rules decide.** A deterministic **rule graph** written in Gosu checks the proposal against **23 rules in 6 layers**. It is code, not a model: the same proposal always gets the same verdict and the same hash. One failure blocks the whole proposal (**all-or-nothing**). The named failure goes back to the planner, which repairs and re-proposes.
3. **People approve.** Only a proposal that passed every rule reaches a named **Compliance Reviewer**, who sees every clause with its provenance and approves or rejects it (a rejection needs a written reason).

Only after approval does the backend build a signed **package**. An **agent** running next to PolicyCenter re-checks the signature, installs the product, restarts PolicyCenter and confirms through PolicyCenter's own API that the product exists.

The one-sentence pitch: ProvenPath lets an AI draft insurance products at machine speed while making it mathematically impossible for an unverified or unapproved configuration to be written into Guidewire PolicyCenter.

Why "pre-commit"

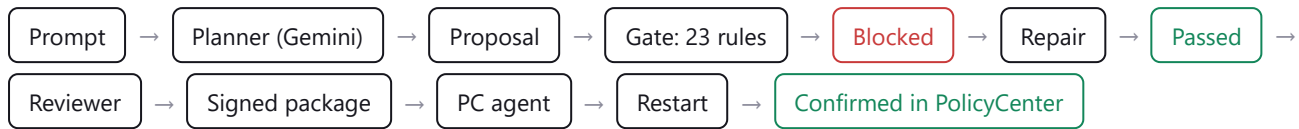
Most AI safety checks look at output *after* it is used. ProvenPath is placed on the *write path*: the only road into PolicyCenter goes through the gate, the reviewer and the signature check. A blocked proposal produces **zero files**; the UI shows "Files written: 0" and "zero files written to PolicyCenter" to make that visible.

Three properties we can prove

Property	What it means	How it is shown
----------	---------------	-----------------

Deterministic	The same proposal + the same rules always produce the same verdict and the same <code>verdictHash</code> .	"Re-verify this run" recomputes the verdict and compares hashes: "Same verdict, same hash".
Provenant	Every approved clause traces back to the rules that checked it and to the exact regulatory text it cites, byte for byte.	The Provenance sheet: clause → rules → cited regulation, "matches the source byte for byte".
Tamper-evident	Changing anything after verification (one word of a citation, one file, one cap) is detected and refused.	Tamper test in the UI; the agent's "Tampered package refused".

The pipeline, end to end



The UI groups these into four stages, shown in the stage rail at the top of Mission Control: **Propose** → **Verify** → **Review** → **Deploy**. Every step emits an **event** (for example `gate.blocked`) that is stored in an append-only event log and streamed live to the browser. The event names are listed with each step below.

Stage 1 · Propose

1. **Request.** The user types a product request, e.g. "Cyber insurance for Indian startups, up to ₹50L coverage". The browser calls `POST /api/v1/executions`. The backend creates an **execution** (one end-to-end attempt, id like `exec-5b35e07c...`) and emits `run.started`.
2. **Planning.** The planner decides what to build and calls tools, exactly as an AI agent would: `propose_product` (the product skeleton: aggregate limit, turnover, minimum premium, effective date), `add_coverage` (the clauses), then `verify_compliance` (hand it to the gate). Events: `planner.step`, `tool.called`, `tool.result`.
3. **Proposal.** The result is a **proposal** (id like `PP-406A3B0F`) with an **iteration** number (1 for the first attempt). Event: `proposal.created`.

Planner modes: **live** = Google Gemini with tool calling; **fixture** = offline, replays two recorded demo proposals (the blocked one, then the fixed one) through the real gate. If Gemini fails in live mode, the planner falls back to the fixture and says so in a `planner.step` with action "fallback".

Stage 2 · Verify

1. **Verification run.** The gate starts a **run** (one verification of one iteration, id = `runId`). Event: `verify.started` with `ruleCount` 23 and `nodeCount` (how many individual checks: rules that apply per clause are evaluated once per clause, so 23 rules become 85 checks for a 10-clause proposal).
2. **Rule by rule.** Rules run in dependency order (a topological sort of the rule graph). Each check emits `verify.node` with a result: PASSED FAILED SKIPPED (a rule it depends on failed, so it could not be judged) or NEEDS_REVIEW.
3. **Verdict.** If every check passed: `gate.passed`, with a **verdict hash** and a **gate token**. Otherwise `gate.blocked` with the named failures and `writtenToPolicyCenter`: `0`. FAILED or NEEDS_REVIEW anywhere means BLOCKED.
4. **Repair loop.** On a block, the planner receives the named failures (rule, clause, expected, actual, reason), fixes the proposal and re-proposes as iteration 2. Event: `planner.repair`. The new iteration goes through the whole gate again.

Stage 3 · Review

1. A passed run triggers `review.requested`, naming the reviewers. Status becomes "Awaiting sign-off".
2. The Compliance Reviewer opens every clause and its provenance, then approves or rejects via `POST /api/v1/reviews`. The backend only accepts a decision on the latest PASSED run, and refuses a rejection without a reason. Event: `review.decided`.

Stage 4 · Deploy

1. **Package.** "Deploy to PolicyCenter" calls `POST /api/v1/deployments`. The backend re-checks everything (approved, PASSED run, stored proposal still hashes to what was verified, rules unchanged, gate token valid) and only then asks the **package builder** for the overlay zip + signed **manifest**. Events: `pc.export`, `pc.queued`.
2. **Pull.** The PC agent on the PolicyCenter machine long-polls `GET /api/v1/pc-agent/next` and claims it. Event: `pc.pulled`.
3. **Verify again, then write.** The agent checks the manifest signature, recomputes the gate token and every file's SHA-256, backs up the current files and writes the new ones into PolicyCenter's `modules/configuration`. Event: `pc.write`.
4. **Restart.** PolicyCenter only reads product-model XML at start-up, so the agent runs `gwb.bat stopServer` and `gwb.bat runServer` (about 4 minutes for a values-only change). Events: `pc.restart` (with elapsed time), `pc.ready`.
5. **Confirm.** The agent asks PolicyCenter's own ProductModelAPI (SOAP) whether the product, each coverage pattern and each capped cov term exist. Event: `pc.verified`, then `run.completed` with status `deployed`. The UI headline becomes "Live in PolicyCenter".

The worked example: the ₹50L cyber product

This is the demo run, with its real numbers (from the recorded run `fixtures/events_demo_run.jsonl` and the fixture proposals).

The request

"Cyber insurance for Indian startups, up to ₹50L coverage"

Iteration 1 · the AI's first proposal (10 clauses)

Clause	Pattern code	Kind · existence	Values	Cites
c-001	SMCyberDataBreachCov	coverage · Required	limit ₹20L, deductible ₹1L	IRDAI §4.1
c-002	SMCyberPrivacyLiabilityCov	coverage · Required	limit ₹20L, deductible ₹1L	IRDAI §4.1
c-003	SMCyberExtortionCov	coverage · Suggested	limit ₹40L , deductible	IRDAI §3.4
c-004	SMCyberBusinessInterruptionCov	coverage · Suggested	limit ₹10L, 12 h waiting period	IRDAI §3.3
c-005	SMCyberRegulatoryFinesCov	coverage · Electable	limit ₹5L	IRDAI §4.4
c-006...009	War, PriorKnown, IntentionalActs, InfraFailure	exclusions · Required	—	IRDAI §4.2
c-010	SMCyberBaseRatingCov	rating · Preset	rating factors	IRDAI §3.6

Proposal-level values: aggregate limit ₹50,00,000; turnover ₹20 Cr; minimum premium ₹25,000.

Gate: BLOCKED on CYB-RNG-002 (layer RANGE). "Extortion sublimit ≤ 50% of aggregate": 50% of ₹50L = ₹25,00,000, but the proposal says ₹40,00,000.

UI: Proposed **₹40,00,000** · Allowed **₹25,00,000** · source IRDAI-CYB-G-2024-S3.4 · Files written: **0**.

CYB-CON-001 ("sum of first-party sublimits ≤ aggregate") is SKIPPED because it depends on CYB-RNG-002.

Repair → iteration 2

The planner receives the named failure and lowers the extortion limit to ₹20,00,000 (within ₹25L). The whole gate runs again: **23/23 rules pass** (85 checks). Event `gate.passed`; a verdict hash and gate token are issued.

Review

"A. Mehta — Compliance Reviewer" sees the 10 verified clauses, opens provenance, and clicks "Approve for PolicyCenter". The approval is logged against the verdict hash.

Deploy

The package contains the SMCyber product and its 3 PolicyCenter coverage patterns (Data Breach, Extortion, Business Interruption), with the verified caps written into PolicyCenter's own term limits:

PolicyCenter cov term	Range written	Comes from
SMCyberExtortionLimit	0 – 25,00,000	CYB-RNG-002 ($\leq$ 50% of aggregate)
SMCyberExtortionDeductible	0 – 2,50,000	CYB-RNG-003 (deductible $\leq$ 10% of that cap)
SMCyberDataBreachLimit	0 – 50,00,000	CYB-CON-001 (no sublimit above the aggregate)
SMCyberDataBreachDeductible	0 – 5,00,000	CYB-RNG-003
SMCyberBILimit	0 – 50,00,000	CYB-CON-001
SMCyberBIWaitingHours	8 – 72 hours	CYB-RNG-004

So even inside PolicyCenter, an underwriter who later types ₹40L into the extortion limit is refused by PolicyCenter's native validation. Clauses that have no PolicyCenter pattern in our overlay (privacy liability, regulatory fines, the four exclusions, base rating) are verified by the gate but not installed; they are listed in `provenpath-report.json` inside the package.

The deterministic gate: 6 layers, 23 rules

Each rule is a YAML file in `rules/rules/` with a code, a layer, what it applies to (the whole proposal, every clause, or one coverage), its logic, the regulation it enforces (`source_code`) and what it depends on. The rules form a **directed acyclic graph** (DAG): a rule only runs after the rules it depends on. Most rules depend on CYB-TYPE-001, because nothing else can be judged if the basic types are wrong. The **ruleset hash** is a SHA-256 over all rule files: if any rule changes, the hash changes, and older approvals can no longer be deployed.

Layer 1 · Type & schema

Code	Rule	What it checks
CYB-TYPE-001	Type field validation	Money fields are non-negative INR integers; pattern codes follow <code>SMCyber...(Cov Exc1)</code> .
CYB-TYPE-002	Owning entity type validation	Every clause is owned by <code>GeneralLiabilityLine</code> (PolicyCenter line pattern <code>GLLine</code>).
CYB-TYPE-003	Existence and category validation	Existence $\in$ {Required, Suggested, Electable, Preset}; a valid category.

Layer 2 · Range & limits

Code	Rule	What it checks
CYB-RNG-001	Aggregate limit range 5L to 5Cr	$\text{₹}5,00,000 \leq \text{aggregate} \leq \text{₹}5,00,00,000$.
CYB-RNG-002	Extortion sublimit $\leq 50\%$ of aggregate	The ransomware/extortion limit is at most half the aggregate.
CYB-RNG-003	Deductible 1–10% of limit	Each coverage's deductible is between 1% and 10% of its limit.
CYB-RNG-004	BI waiting period 8–72 hours	Business-interruption waiting period.
CYB-RNG-005	Turnover $\leq 250\text{Cr}$	The product is for SMEs.
CYB-RNG-006	Rating factors 0.5–3.0	Every pricing factor in the rating clause.
CYB-RNG-007	Minimum premium floor	Minimum premium $\geq \text{₹}10,000$.

Layer 3 · Consistency

Code	Rule	What it checks
CYB-CON-001	Sum of first-party sublimits $\leq$ aggregate	Depends on CYB-RNG-002.
CYB-CON-002	Deductible < limit for each coverage	
CYB-CON-003	No exclusion nullifies a Required coverage	An exclusion may not cancel a mandatory coverage.
CYB-CON-004	No duplicate pattern codes	

Layer 4 · Rule match

Code	Rule	What it checks
------	------	----------------

CYB-RM-001	Mandatory coverages present	Data Breach Response and Privacy Liability must be there.
CYB-RM-002	Mandatory exclusions present	War, prior known incidents, intentional acts, infrastructure failure.
CYB-RM-003	Ransom cover carries law-enforcement-notification condition	On SMCyberExtortionCov.
CYB-RM-004	Fines coverage has insurability condition	Fines only "where insurable by law".
CYB-RM-005	CERT-In 6-hour notification condition present	Cyber incidents must be reported to CERT-In within 6 hours.

Layer 5 · Source integrity

Code	Rule	What it checks
CYB-SRC-001	Every clause has ≥ 1 citation	
CYB-SRC-002	Citation source exists, snippet matches, active on date, jurisdiction IN	The cited text must match the stored regulation byte for byte . This catches paraphrasing.
CYB-SRC-003	Source active on proposal target effective date	No citing a regulation that is not in force on the product's start date.

Layer 6 · Grounding

Code	Rule	What it checks
CYB-GRD-001	Every number in proseSummary matches a verified value	The AI's written summary cannot mention a number (₹, hours, %) that the verified clauses do not contain.

Results and the verdict

Result	Meaning
PASSED	The check holds.
FAILED	The check is violated. Blocks.
SKIPPED	Not evaluated because a rule it depends on failed. Shown dashed in the graph.
NEEDS_REVIEW	The gate cannot decide (e.g. a clause no rule covers, shown as UNMATCHED). Blocks, on purpose.
PENDING	UI only: not evaluated yet in this run.

In the graph, a rule shows the **worst** result across all clauses it checked (FAILED > NEEDS_REVIEW > SKIPPED > PASSED), with "×10" meaning it ran on 10 clauses. The verdict is **PASSED** only if nothing is FAILED or NEEDS_REVIEW; otherwise **BLOCKED**.

The regulatory source set

`rules/sources.yaml` holds 21 source texts (IRDAI-style sections and CERT-In) with code, section, title, full text, effective date and jurisdiction. It is a curated, **illustrative** set written for this demo, labelled as such in the UI;

ProvenPath is a rule-graph check, not legal advice.

Trust and tamper-proofing

Every link in the chain is sealed with a hash or a keyed signature, so that "what was verified" is exactly "what was approved" is exactly "what was installed".

Seal	What it is	Why it exists
proposalHash	SHA-256 of the proposal in canonical JSON (keys sorted, so storage cannot reorder it).	At deploy time the stored proposal is re-hashed; if anyone edited it in the database: "proposal_tampered", nothing deployed.
rulesetHash	SHA-256 over all rule files.	If rules changed since verification: "ruleset_changed", re-verification required.
verdictHash	SHA-256 of the verdict (all node results; excludes the random runId).	Replay proves determinism: same proposal, same rules → same verdictHash. The reviewer's approval is logged against it.
gateToken	HMAC-SHA256 of <code>runId proposalHash rulesetHash</code> with the secret <code>PROVENPATH_GATE_SECRET</code> . Issued only for a PASSED run.	Proof that this exact proposal passed this exact ruleset. Required to build a package; re-checked by the agent.
Manifest signature	HMAC-SHA256 over the whole canonical manifest (file hashes, term ranges, reviewer, token...).	Nobody can change a cap or swap a file after the backend built the package.
File SHA-256s	One per file in the package, listed in the manifest.	The agent refuses a package with any changed, missing or extra file.
Agent key	Bearer token <code>PC_AGENT_KEY</code> .	Only the real agent can pull packages and report deployment steps.

The tamper test (in the ⋮ menu)

Takes the real compliant demo proposal, changes one word of a cited regulation ("shall" → "may", what a paraphrasing model does), and sends it to the real gate (POST `/api/v1/verify`, stateless: nothing stored). Result: BLOCKED at the SOURCE layer (CYB-SRC-002, snippet does not match the source).

What the agent refuses (all tested)

- a file changed inside the zip (sha256 mismatch), a manifest edited to raise a cap (signature fails), a package signed with another secret;
- any path outside the product model (e.g. `../../Windows/...` or Gosu code in `config/gsrc`), even if correctly signed;
- a package that belongs to a different run than the backend said.

In every refusal: "Tampered package refused: ... Nothing was written to PolicyCenter."

From approval to PolicyCenter

The package builder (Gosu module :pcexport)

Starts from `policycenter/overlay-template`, the SMCyber "v0" file set already installed on the Guidewire VM, and fills it from the verified proposal. It never invents structure; it only sets values: each coverage's `existence` and each cov term's `minVal` / `maxVal`. A term range is written only if the rule that governs it PASSED. Contents of the zip:

File	Purpose
<code>products/SMCyber/SMCyber.xml + -lookups.xml</code>	The product "SME Cyber Insurance" (reuses the GL line).
<code>coveragepatterns/SMCyber...Cov.xml + -lookups.xml (×3)</code>	Data Breach, Extortion, Business Interruption with their cov terms and verified limits. Only available when the policy's product is SMCyber (AvailabilityScript).
<code>productmodel.display.properties</code> fragment	The display names PolicyCenter shows, in a marked block.
<code>pc-edits.json</code>	One PCF edit: the GL wizard also serves SMCyber (<code>mode="GeneralLiability SMCyber"</code>).
<code>provenpath-report.json</code>	Which clauses were installed and which have no PolicyCenter pattern.
<code>provenpath-manifest.json</code>	The signed manifest (not listed in itself).

The PC agent (Gosu module :pcagent , one jar)

Runs on the PolicyCenter machine as the normal user, from a console. It **pulls** (long-polls the backend) rather than being pushed to, because the college network and the VM allow no inbound tunnels; the only link between laptops and the VM is GitHub. Its loop for each package:

1. **Verify**: signature, gate token, every file hash, allowed paths only.
2. **Back up** every file it may touch to `BACKUP_DIR/<deployment>-<time>/before/` (and keep the manifest + report).
3. **Write** only files whose bytes change; merge the display-name block (replace in place, keep the file's CRLF line endings); apply the PCF edit idempotently. Any error rolls back from the backup.
4. **Restart** only if something changed (or PolicyCenter is down): `PC_STOP_CMD`, then `PC_START_CMD` as a child process with its console output in `LOG_DIR`, using PolicyCenter's own Java (`PC_JAVA_HOME`). Reports "PolicyCenter restarting · 2m 30s" every 30 s.
5. **Ready** when ProductModelAPI answers again.
6. **Confirm**: `getPublicIdForCodeIdentifier` for the product (PRODUCT), each coverage (CLAUSEPATTERN) and each capped term (COVTERMPATTERN): 10 codes for the demo.

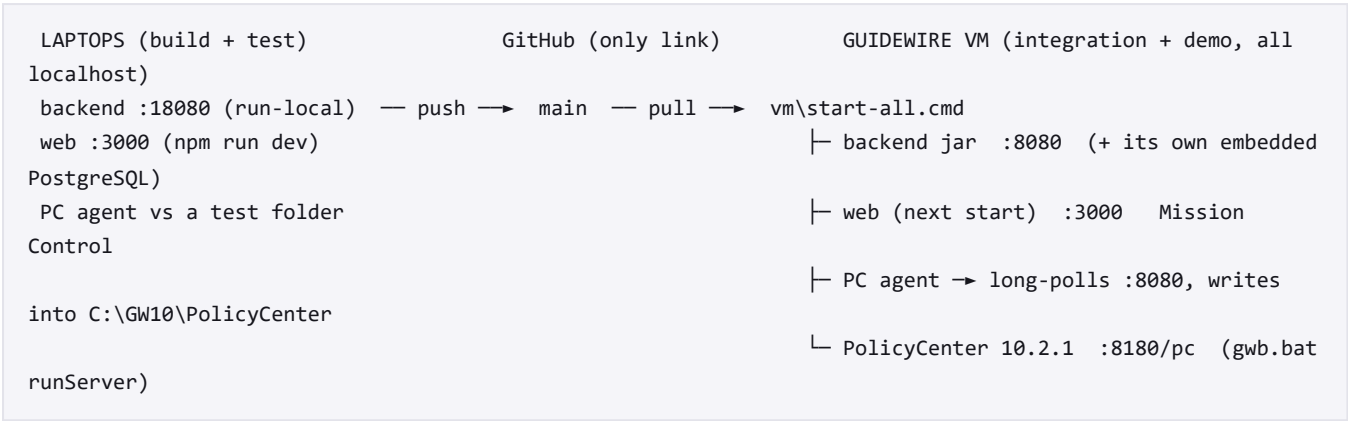
A second deploy of the same product changes nothing and skips the restart. A fallback inbox folder (`INBOX_DIR`) accepts dropped zips too.

Measured PolicyCenter timings (on the Guidewire VM)

Change	Restart time
First install of new pattern files	22 m 17 s (full Gosu compile of ~29,500 classes)
Values-only change (e.g. a new cap)	3 m 54 s
PCF attribute change	5 m 29 s
No change	3 m 31 s

That is why the SMCyber pattern set is pre-installed before a demo: a live deploy then only changes values and takes about 4 minutes.

The system: components and where they run



Backend modules (Gradle multi-module, Gosu 1.18 on Java 11)

Module	Owner / role
:contracts	Shared types: Proposal, Clause, Citation, Verdict, NodeResult, PcManifest, ports (PlannerPort, VerifyPort, PackageBuilderPort), JSON and Signing helpers.
:core	The gate: rule loader (YAML), rule graph (DAG), logic evaluator, hashing, gate token.
:eval	Runs the labelled corpus (eval/corpus , 40 proposals) through the gate → eval/metrics.json .
:app	Javalin HTTP server + SSE, embedded PostgreSQL, Flyway migrations, services (executions, verify, review, deploy, agent, tools, provenance), fixture planner.
:planner	The Gemini planner: tool calling, repair loop, fallback to the fixture.
:pcexport	The package builder (loaded by class name).
:pcagent	The PC agent (its own jar).

Data (PostgreSQL, 9 tables)

Users (proposer, reviewer, admin), executions, proposals, verification runs + per-node results, reviews, deployments (manifest + zip), and the event log, which is **append-only** (a database trigger refuses updates and deletes). On laptops and on the VM the backend starts its own private PostgreSQL 16 (DB_MODE=embedded): nothing to install, no admin rights.

Frontend

Next.js 16 / React 19 in web/ . It never computes a verdict, hash or metric; it only displays what the backend returns. Live progress arrives over **Server-Sent Events** (SSE) from /executions/{id}/stream , with automatic reconnect that resumes from the last event. Animations: Motion and Lenis. An MCP endpoint (/api/mcp) exposes the same four tools to any MCP-capable AI client.

Every screen and every word in the frontend

Landing page (/)

On screen	Meaning
AI proposes. Rules decide. People approve.	The three steps of the product.
PolicyCenter 10 badge	The target system: Guidewire PolicyCenter version 10.
Animated gate (hero)	Illustration: proposals stream toward a gate of six layers; most pass (green), some shatter (red).
gate backend online · ruleset d7762edb · planner live	Live status from <code>/health</code> : backend reachable, first 8 characters of the ruleset hash, planner mode. "offline" means the backend is not running; the recorded run still works.
Launch Mission Control	Opens the app with a real boot check.
Watch a recorded run	Replays a real recorded run (no backend needed).
Rule marquee	The 23 real rule codes and names.
The problem / How it works	Scroll story; the 4 visuals are labelled "from a recorded run".
Six layers. No opinions.	The six layers with their real rules from <code>/rules</code> .
Change one word. Get caught.	The tamper story (CYB-SRC-002).
Measured by the gate itself.	Live eval numbers from <code>/metrics</code> ; "—" if the backend is offline (never hard-coded).

Launch sequence

Line	Real check behind it
Reaching the gate backend	<code>GET /health</code> → shows the planner mode.
Loading the rule graph	<code>GET /rules</code> → "23 rules · 6 layers · <ruleset hash>".
Finding compliance reviewers	<code>GET /users</code> → number of reviewers.

Mission Control (/control): top bar

On screen	Meaning
Stage rail: Propose · Verify · Review · Deploy	Where the current run is. Green = done, highlighted = current, red = stopped there (blocked or rejected).
RECORDED badge	You are watching a replay, not a live run.
gate online / offline	Backend health.
New run	Clear the screen for a new request.
Metrics	Opens "How good is the gate?".

Empty state

On screen	Meaning
What should we insure today?	Type the product request; Enter runs it.
Live run / Recorded run	Real backend run, or replay of <code>fixtures/events_demo_run.json1</code> .
Suggestion chips	Example prompts.
"offline planner: every prompt produces the recorded demo proposal..."	Shown in fixture mode: the prompt does not change the proposal, but the verdict is still computed live.

The stage (left side)

On screen	Meaning
REQUEST · exec-...	The prompt of this execution and its id.
ITERATION 2 · RUN f6fa2bbc	Which proposal attempt, and the first 8 characters of its verification run id.
Headline	Connecting → Starting → Verifying → Blocked → Repairing → Passed → Awaiting sign-off → Approved / Rejected → Deploying → Live in PolicyCenter (or Error).
PASSED · FAILED · SKIPPED · PENDING	Counts of rules in this run (23 total).
Segment bar	One segment per rule, coloured by its result.
01 Type ... 06 Grounding (column headers)	The six layers; the small bar fills as that layer's rules finish, red if one failed.
Rule node "RNG-002 · Extortion sublimit..."	One rule. Dot colour = worst result; "×10" = ran on 10 clauses. Click for details. Lines = dependencies, coloured by the result of the rule they lead into.
Scan sweep	Plays when a new verification run starts.
Stamp "Blocked · zero files written to PolicyCenter"	The gate blocked this iteration; nothing can be deployed from it.
Stamp "Passed · every rule passed · sent for sign-off"	The gate passed; a reviewer is now needed.

The inspector (right side): tab "Now"

Shows one thing at a time, whatever matters now: activity → blocked → sign-off → deployment. Clicking a rule temporarily shows the rule.

View	Words and meaning
------	-------------------

Activity	The story of the run: planner steps (e.g. "Add mandatory coverages, exclusions and rating"), tool calls ("propose_product()"), "Proposal PP-... · iteration 1", "Gate checking 23 rules" with a progress bar of the 85 checks, "Blocked by CYB-RNG-002", "Repairing CYB-RNG-002: got ₹40,00,000, allowed ₹25,00,000", "Passed on iteration 2", "Waiting for compliance sign-off". "+2.37s" = time since the run started.
Blocked	"Iteration 1 · Blocked", "N blocking results", Files written 0 ; per failure: rule code, layer, clause, Proposed (what the AI wrote) vs Allowed (what the rule permits), the reason, the regulation, "Provenance ↗". "Repairing." = the planner is already fixing it.
Rule	Rule code, name, layer, "applies to", source regulation, "Runs after ..." (dependencies), and "Results in this run" per clause: got ... · rule
Compliance review	"Sign-off required", the verdict hash, the reviewer, "Verified clauses" (click = provenance), "Reject" (asks for the compliance reason) / "Approve for PolicyCenter". After approval: "Approved by ...", "The gate token and this approval are now both required to build the package", "Continue to deployment".
Deployment	"Ship to PolicyCenter" → "In PolicyCenter". Steps: Package signed (built only with a valid gate token and approval) · Queued (waiting for the agent) · Pulled (the agent re-verifies the signature) · Written (overlay copied into modules/configuration) · Restarting (gwb runServer, with a timer) · Ready · Confirmed (ProductModelAPI returns the SMCyber patterns). Buttons: "Deploy to PolicyCenter", "Package" (download the zip), "Open PolicyCenter". "Signed manifest · 11 files · <hash>" lists every file and its SHA-256.

Tab "Trace"

Every raw event exactly as the backend sent it (sequence number, type; click to see the JSON payload). The engineer's view.

Overlays

Overlay	Words and meaning
Provenance sheet	For one clause: name, pattern code, kind, existence, owning entity; Limit / Deductible / Waiting; "Proposal PP-..., iteration 2"; "Checked by N rules · run ... PASSED" with each rule's result; "Cited regulation" with the source code, "matches the source byte for byte" (or "does not match"), title, section, jurisdiction, effective date, and the quoted text.
Metrics: "How good is the gate?"	False passes = non-compliant proposals the gate wrongly passed (0 / 20, the target is always 0); False blocks = compliant proposals wrongly blocked (0 / 20); Accuracy = share of correct verdicts; Provenance = share of clauses with a matching citation; per-layer bars; "Manual configuration ~3 weeks" (PRD estimate, not measured) vs "This run: prompt → verified" (measured from this run's events).
Re-verify toast	"Same verdict, same hash" + original → recomputed hash; "rules changed since" if the ruleset differs.
Tamper toast	"Tamper caught · N blocking results · nothing stored".

Insurance and Guidewire glossary

Term	Meaning
Guidewire PolicyCenter	The core system insurers use to define products and write policies. We use PolicyCenter 10.2.1 on a Guidewire-provided VM at <code>localhost:8180/pc</code> .
Product model	PolicyCenter's configuration of what can be sold: products, lines, coverages, terms. Stored as XML under <code>config/resources/productmodel</code> and read at start-up.
Product (SMCyber)	"SME Cyber Insurance", our product code.
Policy line / GLLine	A line of business inside a product. SMCyber reuses the General Liability line (pattern <code>GLLine</code> , entity <code>GeneralLiabilityLine</code>).
Coverage pattern	The template of a coverage (e.g. <code>SMCyberExtortionCov</code>). One XML file + a lookups file.
Cov term	A value inside a coverage: a limit, a deductible, a waiting period.
CovTermLimits minVal / maxVal	Bounds PolicyCenter itself enforces on a term; ProvenPath writes the verified caps here.
Existence	Required (always on), Suggested (on by default), Electable (customer may add), Preset (fixed).
Aggregate limit	The most the policy pays in total per policy period (₹50L in the demo).
Sublimit	A lower cap for one coverage inside the aggregate (extortion $\leq$ ₹25L).
Deductible	The amount the insured bears before the insurer pays.
Waiting period	For business interruption: hours of downtime before cover starts (8–72 h).
Exclusion	What the policy does not cover (war, prior known incidents, intentional acts, infrastructure failure).
First-party / third-party	Losses of the insured itself (breach response, extortion, BI) vs claims by others against it (privacy liability).
Turnover	The company's annual revenue ($\leq$ ₹250 Cr: this is an SME product).
Minimum premium / rating factors	The lowest price and the multipliers used to price the policy.
IRDAI	Insurance Regulatory and Development Authority of India (our sources are IRDAI-style and illustrative).
CERT-In	India's cyber emergency team; incidents must be reported within 6 hours.
ProductModelAPI	PolicyCenter's SOAP web service (version pc1000) to query the product model; <code>getPublicIdForCodeIdentifier</code> answers "does this code exist?".
SOAP	An XML web-service protocol; PolicyCenter 10 has no product-model REST API.
gwb.bat	PolicyCenter's build/run tool: <code>runServer</code> , <code>stopServer</code> .
PCF	Page Configuration Format: PolicyCenter's screen definitions (we change one attribute so SMCyber uses the GL screens).
Display properties	The file with human names for product-model codes.

Overlay	Our files added on top of PolicyCenter's base configuration, without editing base product files.
Underwriter / Compliance reviewer	The person who prices and accepts risks / the person accountable for regulatory compliance of the product.

Technical glossary

Term	Meaning
LLM / planner	The language model and the code that drives it; here Google Gemini (<code>gemini-3.8-flash</code>) with tool calling (the model asks to call named functions with structured arguments).
Tools	<code>propose_product</code> , <code>add_coverage</code> , <code>verify_compliance</code> , <code>deploy_product</code> (the last one is never offered to the model). Schemas in <code>shared/tools</code> .
MCP	Model Context Protocol: a standard so any AI client can call our four tools at <code>/api/mcp</code> .
Fixture	A recorded input or run used offline (the demo proposals, the recorded event log).
Execution · proposal · iteration · run	One end-to-end attempt · one product configuration · the attempt number within an execution · one verification of one iteration.
Deterministic	Same input → same output, every time (no randomness, no model).
DAG / topological order	Rules with dependencies and no cycles; evaluated so that dependencies come first.
Hash (SHA-256)	A fingerprint of data; any change gives a different hash.
HMAC	A hash keyed with a secret; only holders of the secret can produce or check it.
Canonical JSON	JSON written in one fixed form (sorted keys) so its hash is stable.
Gosu	Guidewire's JVM language; our whole backend, gate, builder and agent are written in it (Gosu 1.18).
Javalin / SSE	The HTTP server library / Server-Sent Events, a one-way live stream from server to browser.
Long-poll	The agent asks "anything for me?" and the server holds the request up to ~30 s until there is.
Embedded PostgreSQL	A private database the backend starts itself; no install, no admin.
Append-only event log	Events can be added but never edited or deleted (enforced by a database trigger): the audit trail.
Eval corpus	40 labelled proposals (20 compliant, 20 not) used to measure the gate.

Reference: events and API

Events (stream order for one execution)

Event	Payload (main fields)
run.started	prompt, mode (fixture live mcp test)
planner.step	step, action, note, mode
tool.called / tool.result	tool, args / result
proposal.created	proposalId, iteration, clauses, aggregateLimitNr
verify.started	runId, iteration, proposalId, ruleCount (23), nodeCount, rulesetHash
verify.node	runId, ruleCode, clauseId (null = whole proposal), layer, result, expected, actual, reason, sourceCode
gate.blocked	runId, iteration, verdictHash, failedRules, skippedRules, needsReviewClauses, failures, writtenToPolicyCenter: 0
gate.passed	runId, iteration, verdictHash, proposalHash, rulesetHash, nodeCount
planner.repair	runId (blocked run), iteration (next), failedRule, clauseId, expected, actual, reason
review.requested / review.decided	runId, iteration, verdictHash, reviewers / reviewId, decision, reviewer, comment
pc.export / pc.queued	deploymentId, runId, productCode, files, packageBytes, manifestSha256 / deploymentId
pc.pulled	deploymentId, agent
pc.write / restart / ready / verified	deploymentId, detail (from the agent)
pc.failed	deploymentId, detail (or stage: export)
run.completed	status (blocked rejected deployed error)

Backend API (/api/v1)

Endpoint	Purpose
GET /health	Status, ruleset hash, planner, package builder.
GET /rules · /users · /metrics	The rule graph · reviewers · eval numbers.
POST /executions · GET /executions/{id}	Start a run · its runs, reviews, deployments, latest proposal.
GET /executions/{id}/stream	SSE live events.
POST /executions/{id}/replay	Re-verify; compare verdict hashes.
POST /verify	Stateless verification of any proposal (tamper test).
POST /tools/{name}	The four tools (used by MCP).
POST /reviews	Approve / reject.

POST /deployments · GET /deployments/{id}/package]	Build + queue the package · its manifest / zip.
GET /pc-agent/next · POST /pc-agent/status	Agent pulls work · reports steps (bearer PC_AGENT_KEY).
GET /provenance/{clauseId}	Clause → rules → citations.
POST /admin/shutdown	Clean stop, localhost only.

What is real today, what is not, and how to run it

Real and tested

- Backend, the 23-rule gate, hashes, gate token, reviews, replay, provenance, event log: **62 backend tests pass** (core 34, app 12, eval 1, package builder 7, agent 8).
- Eval: 40 labelled proposals, **0/20 false passes, 0/20 false blocks**.
- Package builder: real PolicyCenter XML from the installed template, signed.
- PC agent: the full loop (verify, back up, write, restart, confirm, refuse tampering), tested end to end with the real backend and UI.
- SMCyber v0 is installed in the real PolicyCenter on the VM (product + 3 coverages + 6 terms, confirmed via ProductModelAPI).

Not yet, or not real on a laptop

- **On a laptop, "PolicyCenter" is a stand-in** (a test folder and a small script); only the VM has the real one. The first real deploy through the agent into `C:\GW10\PolicyCenter` is the next VM task.
- **Gemini**: the key works and two blocking bugs were fixed (tool schema, retired model). A successful live run with `gemini-3.8-flash` still has to be observed. If Gemini fails, the planner falls back to the recorded proposals and says "fallback" in the activity.
- Regulations are illustrative; the reviewer has no login; PolicyCenter has no INR currency (rupee amounts sit in the USD term limits); rating/quoting in PolicyCenter is not configured.

How to run on a laptop

```
Terminal 1 (backend :18080)  scripts\run-local.cmd --no-build
Terminal 2 (web :3000)       cd web && npm run dev
Terminal 3 (agent, optional) PowerShell: load .env, then
                             java -jar backend\pcagent\build\libs\provenpath-pcagent.jar
policycenter\agent\agent.env
```

How to run on the Guidewire VM

```
git pull
copy .env.example .env                                     (fill the secrets, GEMINI_API_KEY)
copy policycenter\agent\agent.env.example policycenter\agent\agent.env
vm\start-all.cmd --build      backend :8080, web :3000, PC agent, each in its own console
vm\stop-all.cmd               stops ProvenPath; PolicyCenter keeps running
```

Honesty rule we follow: the UI never shows a number, verdict or step that the backend or the agent did not produce. Anything illustrative is labelled ("from a recorded run", "illustrative", "PRD estimate, not a measurement").